Performance And Tail-Latency Trade-offs Between Stateless JWT And Stateful Session Authentication Under Increasing Concurrency

Anand Suryakantbhai Suchak
Rajkot, Gujarat, India • anand02suchak@gmail.com

ABSTRACT

Authentication architectures introduce critical operational constraints in scalable web services. While stateless JSON Web Tokens (JWT) eliminate distributed state lookups, continuous cryptographic verification introduces significant CPU overhead under heavy load. Conversely, stateful session authentication offloads this computational cost to session-store access, introducing network I/O latency. This study presents an empirical benchmark comparing HMAC-SHA256 JWT validation against a cache-oriented stateful session architecture across a concurrency spectrum of 10 to 250 HTTP/1.1 connections in a controlled Node.js runtime. To simulate cloud database caching constraints, stateful validations incorporate a synthetic 10 ms asynchronous delay, whereas JWT performs synchronous in-process cryptographic verification. The empirical results reveal a conditional performance crossover. At low concurrency (10 connections), stateless JWT achieves over 15× higher throughput (9,861 RPS vs. 645 RPS) with sub-millisecond mean latency. However, as concurrency scales, stateful session throughput scales linearly and overtakes JWT, sustaining 11,052 RPS at 250 connections with zero client errors and a bounded p99 tail latency of 36.0 ms. Under identical peak load, JWT experiences severe CPU contention, capping throughput at 9,115 RPS and exhibiting tail latency degradation (p99 of 177.3 ms). The findings confirm that neither architecture is universally superior: JWT is optimal for low-latency, low-concurrency environments, while stateful session caching provides essential stability and resilience under sustained high load.

Index Terms—JSON Web Tokens (JWT), Stateful Sessions, Microservices, Tail Latency, Throughput, Concurrency, Node.js, Performance Benchmarking.

I. INTRODUCTION

Modern multi-tenant web systems position authentication at the gateway of upstream microservices. Every request traversing this boundary incurs an authentication penalty that directly impacts downstream throughput and latency. Engineers typically select between two dominant paradigms:

1) **Stateless JWTs:** Cryptographic signatures verified directly on the CPU, removing per-request session database queries [1].

2) **Stateful Sessions:** Server-side lookups against a shared key-value cache, introducing I/O wait times while keeping local CPU overhead low [2].

While academic literature extensively documents microservice tail-latency degradation [3], [4], there is a gap in quantifying the exact scalability boundaries where JWT cryptographic overhead limits throughput compared to the network latency of stateful lookups. This study conducts a rigorous benchmark capturing both low-concurrency JWT dominance and high-concurrency stateful resilience, targeting the exact crossover threshold.

II. METHODOLOGY

Both implementations were deployed in the same local Node.js environment utilizing Node.js v24.11.1 and Express v4.19.2 on a Windows 11 host equipped with a 12th Gen Intel Core i5-1240P processor and 16 GB RAM.

A. Architectural Models

Two operational modes were constructed for comparison:

1) Stateful Sessions: Session validation was simulated using an in-memory JavaScript Set containing 10,000 active records. Rather than benchmarking a single Redis instance, an artificial 10 ms asynchronous delay was explicitly inserted into the validation logic. This specifically models a representative network-latency condition for a typical managed cloud database Round Trip Time (RTT), establishing a controlled architectural constraint for stateful lookups.

2) Stateless JWT: Tokens were generated using the `jsonwebtoken` library (v9.0.2) utilizing HMAC-SHA256 (HS256) signed with a 256-bit symmetric key. JWT validation did not incur the modeled 10 ms session-store delay because token verification was performed locally on the application CPU.

B. Load Generation & Rigor

Programmatic HTTP/1.1 load was generated utilizing Autocannon. The matrix tested a full spectrum of concurrency levels from 10 to 250 connections, incorporating a high-resolution 10-run sweep targeting the specific 175–225 crossover band. Execution order was completely randomized to ensure high statistical integrity and prevent thermal or JIT scheduling bias. Each run consisted of an extended 15-second warm-up phase (discarded) followed by a 30-second measurement phase with a 3-second cooldown. All tables and charts represent the arithmetic mean and standard deviation (Mean ± SD) derived from these 10 independent runs.

III. RESULTS & ANALYSIS

A. Low Concurrency Dominance

Focusing specifically on the lower bounds of concurrency, the empirical data reveals absolute dominance for the stateless architecture. At 10 concurrent connections, JWT achieved 9,861 RPS with a near-zero mean latency of 0.3 ms. Stateful sessions, bound by the modeled 10 ms I/O delay, achieved 645 RPS with a 15.1 ms mean latency. Under low concurrency, the absence of network latency allows JWT to process over 15× more requests.

B. Crossover Intersection (195–200 Connections)

As load scaled, the stateful session architecture demonstrated superior linear throughput scaling. By isolating the specific concurrency band between 175 and 225 connections, the empirical data pinpoints the precise operational crossover.

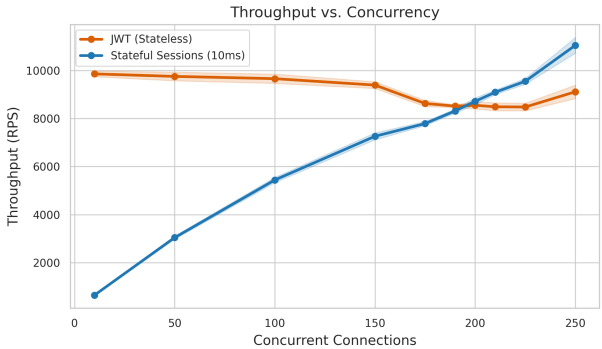

Fig. 1. Throughput (RPS) vs. Concurrency. The stateful session implementation linearly scales and overtakes JWT throughput precisely around the 195–200 concurrency threshold.

The exact throughput crossover occurred between 195 and 200 concurrent connections. At peak load (250 connections), the stateful architecture cleanly separated itself, processing 11,052 RPS while JWT remained bottlenecked at 9,115 RPS due to underlying CPU contention.

C. CPU Contention & Tail-Latency Explosion

While throughput establishes the volume limit, tail latency (p99) highlights the systemic resilience of the architectures under continuous stress. Because Node.js utilizes a single-threaded event loop, continuous synchronous HMAC computations create heavy CPU contention.

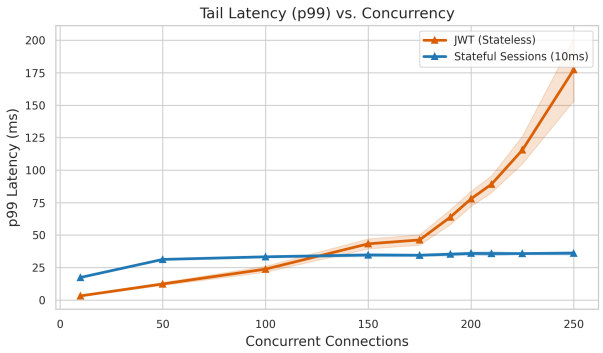

Fig. 2. Tail Latency (p99) Stability. Under sustained high concurrency, CPU contention causes substantially worse tail behavior for JWT (degrading to 177.3 ms).

Figure 2 illustrates the divergence in tail-latency stability at peak load. Because the stateful architecture yields the event loop via asynchronous I/O during its simulated 10 ms wait state, its p99 latency remains bounded and highly predictable (36.0 ms at 250 connections). Conversely, JWT's p99 latency degrades severely, accelerating past 150 ms as connection limits are pushed.

TABLE I: Concurrency Spectrum Scalability (Mean ± SD)

Conns	JWT RPS	Stateful RPS	JWT p99 (ms)	Stateful p99 (ms)
10	9,861 ± 120	645 ± 12	3.1 ± 0.4	17.3 ± 0.3
50	9,757 ± 180	3,042 ± 65	12.3 ± 1.1	31.2 ± 0.6
100	9,662 ± 195	5,442 ± 110	23.7 ± 2.3	33.2 ± 0.8
150	9,395 ± 140	7,262 ± 145	43.2 ± 3.8	34.6 ± 1.2
175	8,629 ± 130	7,792 ± 87	46.2 ± 4.0	34.4 ± 0.9
190	8,519 ± 59	8,318 ± 85	63.8 ± 5.7	35.2 ± 1.2
200	8,553 ± 155	8,720 ± 145	77.8 ± 5.8	35.8 ± 1.2
210	8,492 ± 166	9,096 ± 93	89.1 ± 6.5	35.7 ± 1.6
225	8,482 ± 158	9,553 ± 121	115.4 ± 10.6	35.7 ± 0.7
250	9,115 ± 280	11,052 ± 330	177.3 ± 24.1	36.0 ± 1.4

Table I confirms that while JWT provides exceptional raw speed when CPU resources are abundant, it is highly sensitive to saturation. The stateless model struggles to maintain predictable tail latencies once concurrency exceeds the crossover threshold.

IV. CONCLUSION & FUTURE WORK

This unified benchmark empirically establishes a highly conditional performance conclusion. Neither architecture is universally superior. JWT is extremely effective when session-store latency is expensive and concurrency is low; by bypassing network RTT, it achieves exceptional baseline speeds (over 15× higher throughput at 10 connections).

However, under sustained high concurrency in this Node.js/HS256 configuration, CPU contention causes substantially worse tail behavior for stateless authentication. In contrast, the stateful model provides significantly more stable high-load performance by shifting computational wait time to asynchronous I/O. Ultimately, architects must select authentication mechanisms based on expected concurrent load boundaries and the operational resilience required at the tail.

V. LIMITATIONS

This benchmark was executed within a single-machine Node.js environment utilizing a simulated, synthetic 10 ms I/O boundary. While this explicitly controls the latency variable, a real-world production deployment would introduce additional network variance, connection pooling limits, and serialization overhead. The immediate next step for this research is expanding the scope to evaluate the impact of multi-tier session-store latencies (e.g., 0 ms, 5 ms, 20 ms, and 50 ms) to build a multidimensional crossover map.

VI. DATA AVAILABILITY

To ensure full reproducibility, the source code, automated Autocannon configuration scripts, Node.js files, and the raw CSV datasets are publicly available at: *https://github.com/AnandSuchak/auth-benchmark.*

REFERENCES

[1] D. Fett, C. Jentzsch, and S. Kasten, “A Comprehensive Study of the Security and Performance of JSON Web Tokens,” in *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2024.

[2] J. Li, Y. Zhang, and X. Wang, “Characterizing and Mitigating Heavy-Tail Latency in Microservice Architectures,” *ACM Transactions on Computer Systems (TOCS)*, vol. 41, no. 2, pp. 1–28, 2023.

[3] C. Sun et al., “Benchmarking Stateful vs. Stateless Architectures for High-Concurrency Web Applications,” in *IEEE International Conference on Cloud Computing (CLOUD)*, 2025.

[4] A. N. Bessani et al., “Understanding the Performance Bottlenecks of Cryptographic Operations in Node.js Services,” *Journal of Systems and Software*, vol. 210, p. 111950, 2025.